

Esercitazione 7: Sistema UWB RFID multi-tag

EDOARDO NORI-0001123033

Si consideri lo script MATLAB fornito `RFID_template.m` che implementa un sistema RFID UWB con:

- Modulazione backscatter
- Codici di spreading ortogonali
- Ricevitore a correlazione

Completare lo script inserendo le parti mancanti (indicate come TODO) e rispondere alle seguenti domande.

Parte TAG: Completare (1) il codice per generare i codici Hadamard per l'ID del tag; (2) il codice per la generazione dei dati; (3) lo spreading dei dati.

```
Incollare qui il codice (font Courier size 8)

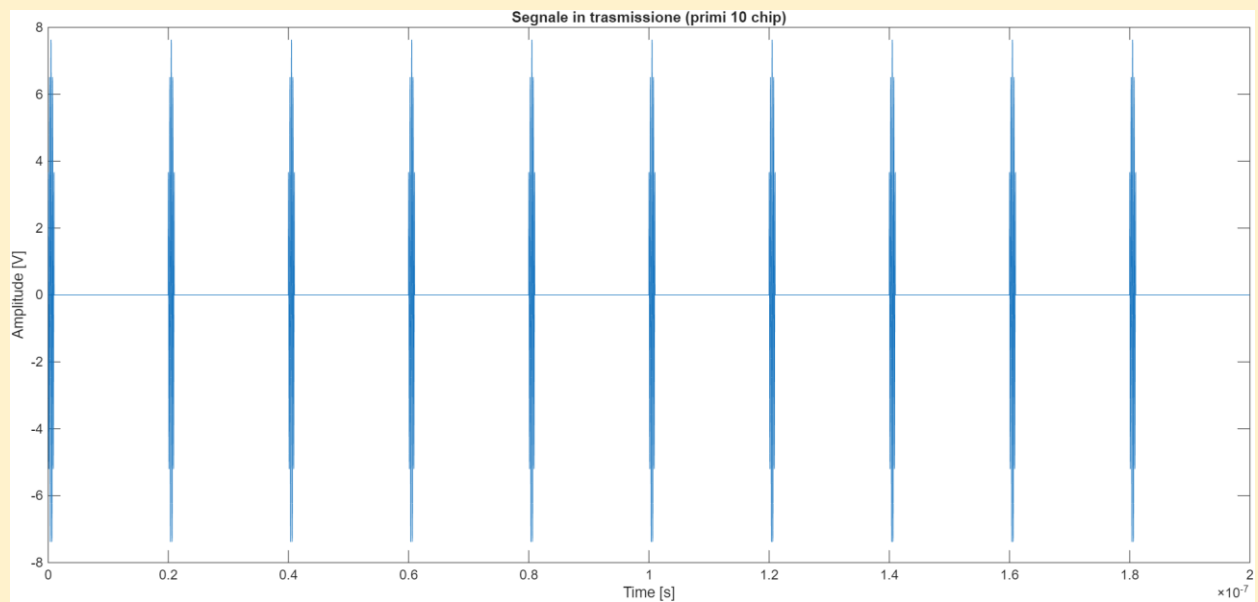
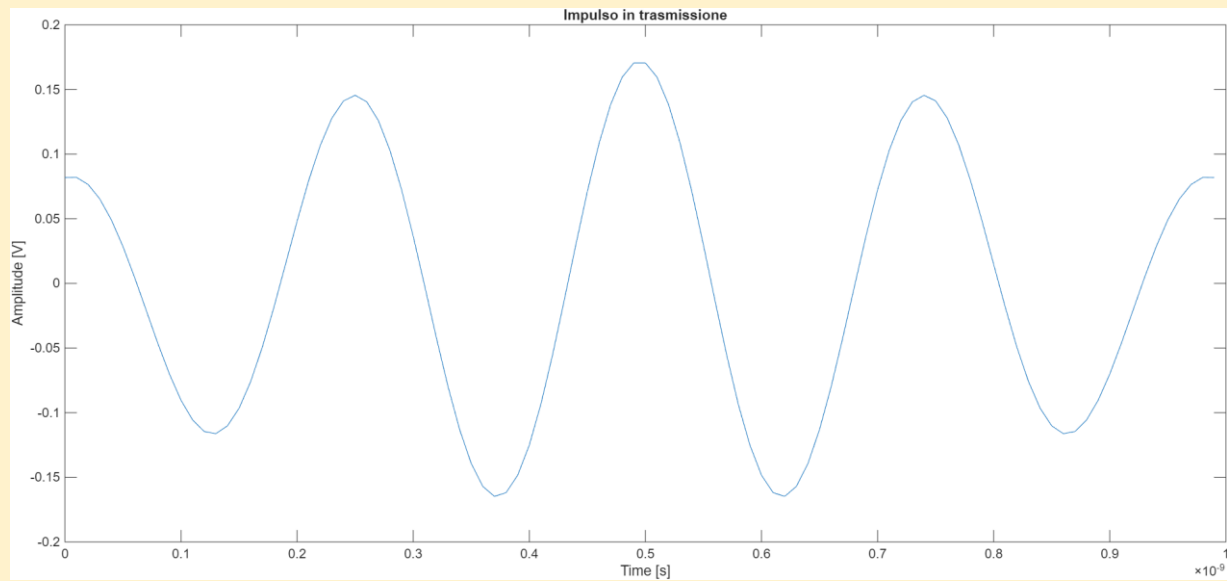
- Spiegare perché i codici devono essere ortogonali
- Perché ogni bit viene "espanso" su più chip?
- Spiegare perché viene fatto l'upsampling - qual è il ruolo di Nsamples_chip?
L'ortogonalità dei codici risolve il problema dei tag multipli.
Ogni bit viene espanso su più chip per aumentare il SNR.
Nsamples_chip è il numero di impulsi UWB per ogni elemento di readerChips.

% TAG
%% =====
for k=1:Ntag
    % Sistema SINCRONO - codici ortogonali
    % TO DO: generare codici Hadamard per tag
    code_tag(k,:) = Hadamard(k, Nc);
% sequenza di 0 e +1
%% =====
    % GENERAZIONE DATI DEI TAG
    %% =====
    % TO DO: generare bit di informazione (usare randi)
    tag_bits(k,:) = randi([0, 1], 1, Nr);
% generazione dati tag k
    % TODO: conversione bipolare {-1, 1}
    tag_bip = 2*tag_bits(k, :) - 1;
% Tag 1: conversione {0,1} -> {-1,+1}
    fprintf('Tag vero: %s\n', num2str(tag_bip));
    % TO DO: spreading : moltiplicazione dell'ID del tag con il suo codice
    tag = [];
    for r=1:Nr
        tag = [tag code_tag(k, :)*tag_bip(k, r)];
% moltiplicazione dell'ID del tag con il suo codice
    end
    c_tag(k,:) = repelem(tag, Nsamples_chip);
ricorda: 1 chip "dura" Nsamples_chip
end
```

Parte Segnale TX: Completare (1) il codice per generare l'impulso `rrcp`; (2) il codice per generare il segnale in trasmissione `tx` (usare la funzione `conv`).

Incollare qui il codice della funzione (font Courier size 8)

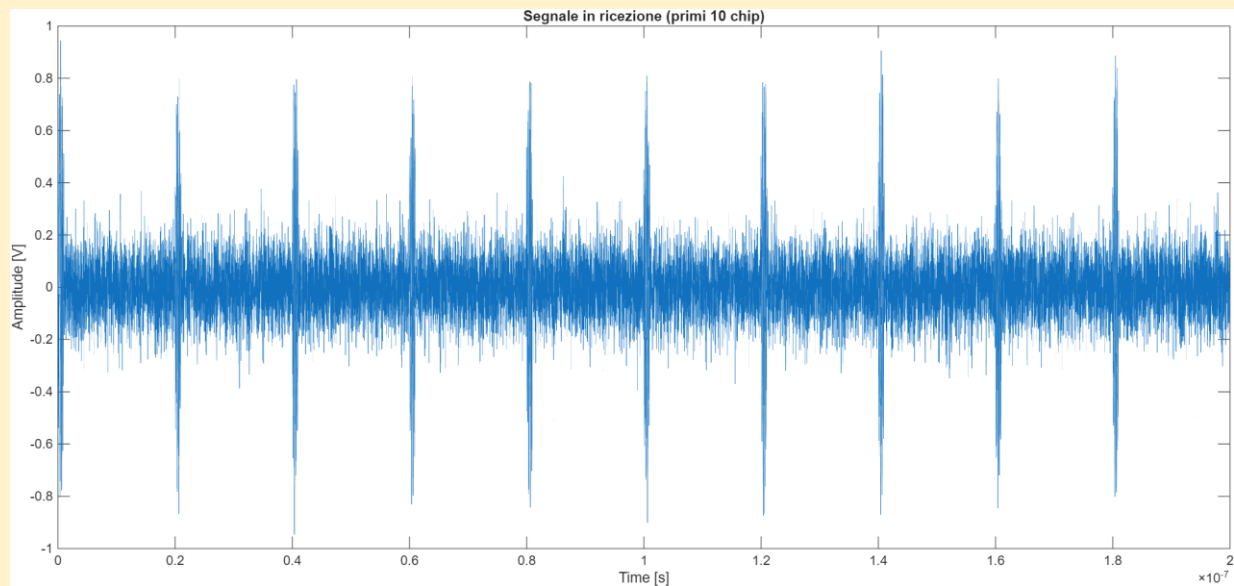
- Plottare l'impulso in trasmissione
- Plottare il segnale in trasmissione. Si plottino i primi $N_{\text{chip_plot}}=10$ chip.



Parte Segnale RX: Completare (1) il codice per modellare il segnale riflesso da ciascun tag; (2) il codice per generare il segnale ricevuto come somma dei contributi di ciascun tag in assenza di rumore; (3) il codice per generare il rumore; (4) il codice per generare il segnale ricevuto.

Incollare qui il codice della funzione (font Courier size 8)

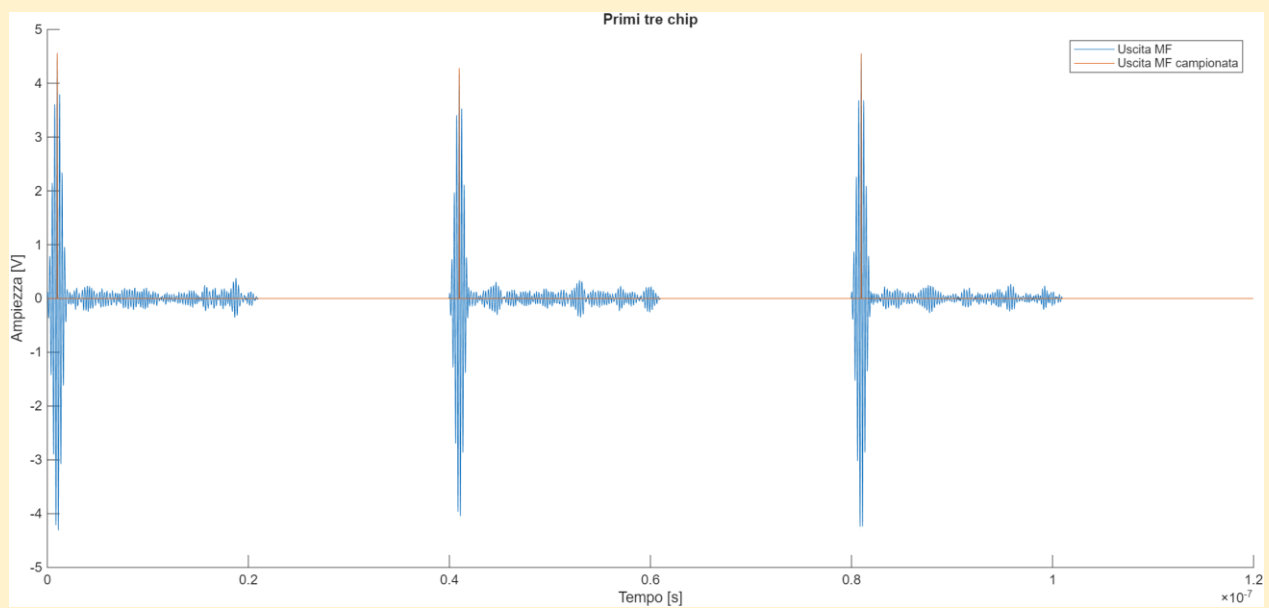
- Plottare il segnale ricevuto. Si plottino i primi $N_{\text{chip_plot}}=10$ chip.



Parte RICEVITORE: Completare (1) il codice per realizzare il filtro adattato; (2) per campionare a livello di chip; (3) per realizzare il despreading (correlatore)

Incollare qui il codice della funzione (font Courier size 8)

- Plottare l'uscita del filtro adattato e sovrapporre i campioni all'uscita del filtro. Si considerino i primi 3 chip.
- Verificare che i campioni corrispondano ai picchi del segnale all'uscita del matched filter

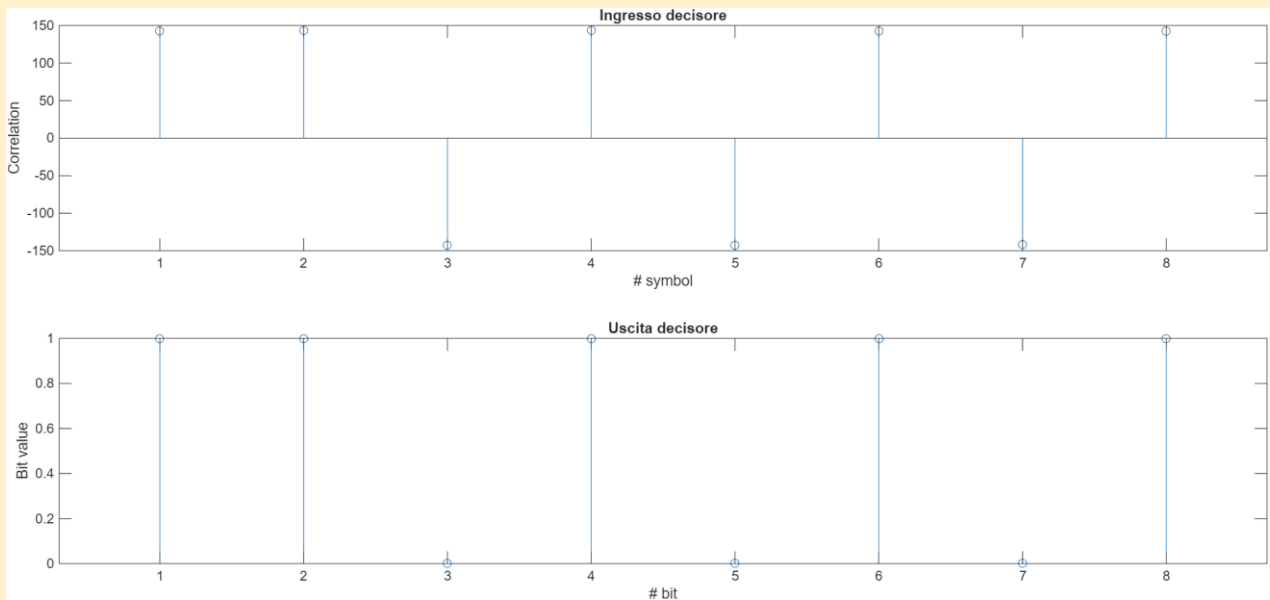


È stato aggiunto uno zero-padding tra un campione e l'altro per facilitare la sovrapposizione tra l'uscita del matched filter e i campioni.

Parte RICEVITORE/DECISIONE: Completare il codice per realizzare una funzione che prenda una decisione sulla sequenza dei bit dati in trasmissione.

Incollare qui il codice della funzione (font Courier size 8)

- Plottare l'ingresso e l'uscita del decisore per ogni tag (usare stem).



Istruzioni:

1. Grafici e risultati prodotti devono recare sempre le relative unità di misura (ms, Hz).
2. Fare le opportune sostituzioni nelle parti evidenziate in verde prima della consegna.
3. Rinominare questo file sostituendo nome e cognome.
4. Caricare su Virtuale entro la scadenza comunicata a lezione:
 - questa relazione compilata in ogni sua parte in **FORMATO PDF**
 - tutto il codice sviluppato (script, funzioni) e i file generati (**.zip o .rar**)
 - il file RFID_cognome_nome.m

Attenzione: non usare **MAI** la funzione “screenshot” del vostro SO. Salvare **SEMPRE** l'immagine con il comando `print()` posizionato dopo il `plot()` con una risoluzione di almeno 300dpi (uno qualunque di questi formati va bene):

```
print(gcf, 'nomefile.png', '-dpng', '-r300');  
print(gcf, 'nomefile.jpg', '-djpeg', '-r300');  
print(gcf, 'nomefile.pdf', '-dpdf', '-r300');
```

oppure, ancora meglio, in formato vettoriale come l'Encapsulated PostScript (EPS):

```
print(gcf, 'nomefile.eps', '-deps');
```

Good practice

Comment your code; describe each module with the following information:

- Name and purpose of the Module
- Description of the Module (in brief)
- Example of usage: input and output
- Author, date and version

Example in Matlab:

```
function [mymod] = mymodexp(b,e,n)  
  
    % mymodexp(b,e,n) gives b^e mod n, where b, e, n are integers  
    %
```

```
% iterative implementation based on the property
%  $(a (a^m \bmod n)) \bmod n = a^{(m+1)} \bmod n$  to avoid numerical overflow
%
% example of usage: mymodexp(137,56,14731) gives as output 6849
%
% Andrea Giorgetti, Oct. 5, 2020, version 1.2 %
```
